

BASIC CHATBOT

Submitted by:

Name: Saikam Bhavya Sri

College: Vignan's Nirula Institute of Technology and Science for Women

Department: Electronics and Communication Engineering (ECE)

1. Abstract

A chatbot is a computer program designed to simulate conversation with users. This project implements a simple rule-based chatbot using Python. The chatbot responds to predefined user inputs such as greetings, questions about its status, and exit commands. The system uses conditional statements and loops to process user input and generate appropriate responses. This project demonstrates the basic concepts of artificial intelligence and human-computer interaction.

2. Introduction

Chatbots are widely used in customer support, virtual assistants, and educational applications. A rule-based chatbot works by matching user inputs with predefined responses. Unlike advanced AI chatbots, it does not learn from conversations but follows fixed rules. The objective of this project is to create a simple chatbot that can interact with users through text-based communication using Python programming.

3. Objectives

- To understand the basics of chatbot development.
- To implement a simple conversational interface.
- To learn the use of conditional statements and loops in Python.
- To provide predefined responses based on user input.
- To demonstrate basic artificial intelligence concepts.

4. Problem Statement

Design and develop a simple rule-based chatbot that accepts user inputs and provides predefined responses. The chatbot should continue the conversation until the user enters an exit command.

5. Scope of the Project

The chatbot can:

- Respond to greetings such as "hello".
- Answer simple questions like "how are you".
- End the conversation when the user enters "bye".
- Handle unknown inputs with a default response.

Limitations:

- Works only with predefined commands.
- Does not understand complex sentences.
- Does not learn from previous conversations.

6. Requirements

Hardware Requirements

- Computer/Laptop
- Minimum 4 GB RAM
- Keyboard and Monitor

Software Requirements

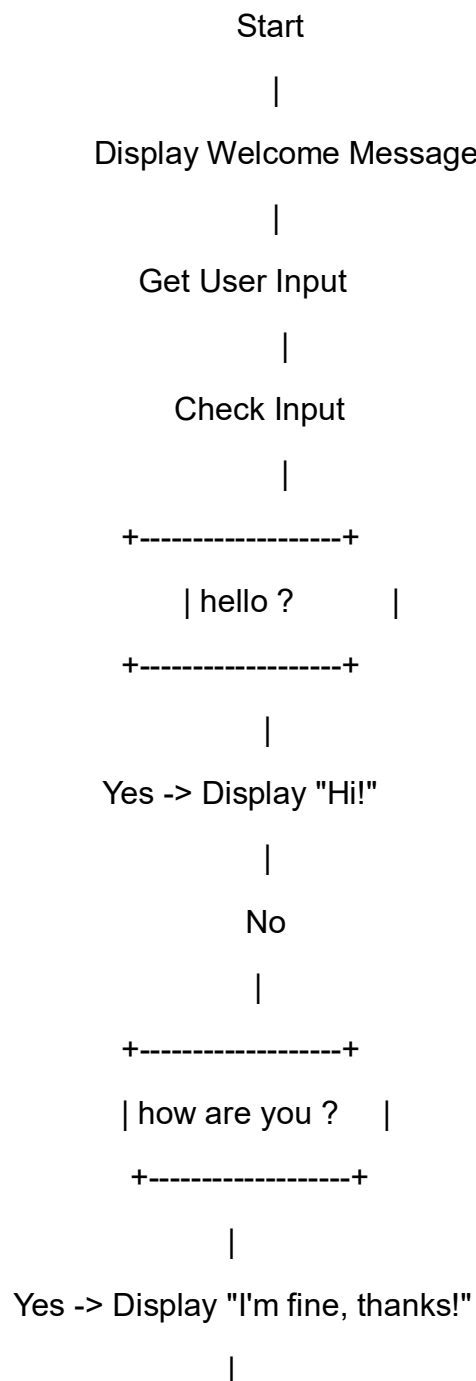
- Python 3.x
- Any Python IDE (IDLE, VS Code, PyCharm, etc.)
- Windows/Linux/Mac Operating System

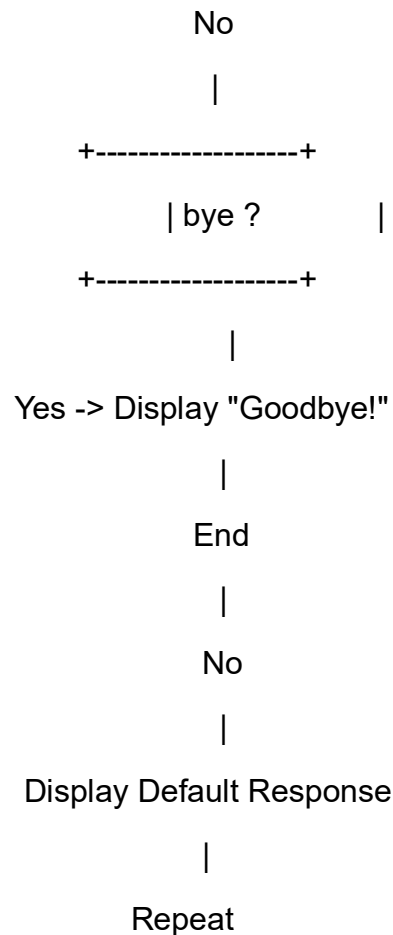
7. System Design

Working Principle

1. The chatbot starts and displays a welcome message.
2. The user enters a message.
3. The chatbot checks the input using if-elif conditions.
4. A corresponding response is displayed.
5. The process repeats until the user enters "bye".
6. The chatbot terminates the conversation.

Flowchart





8. Algorithm

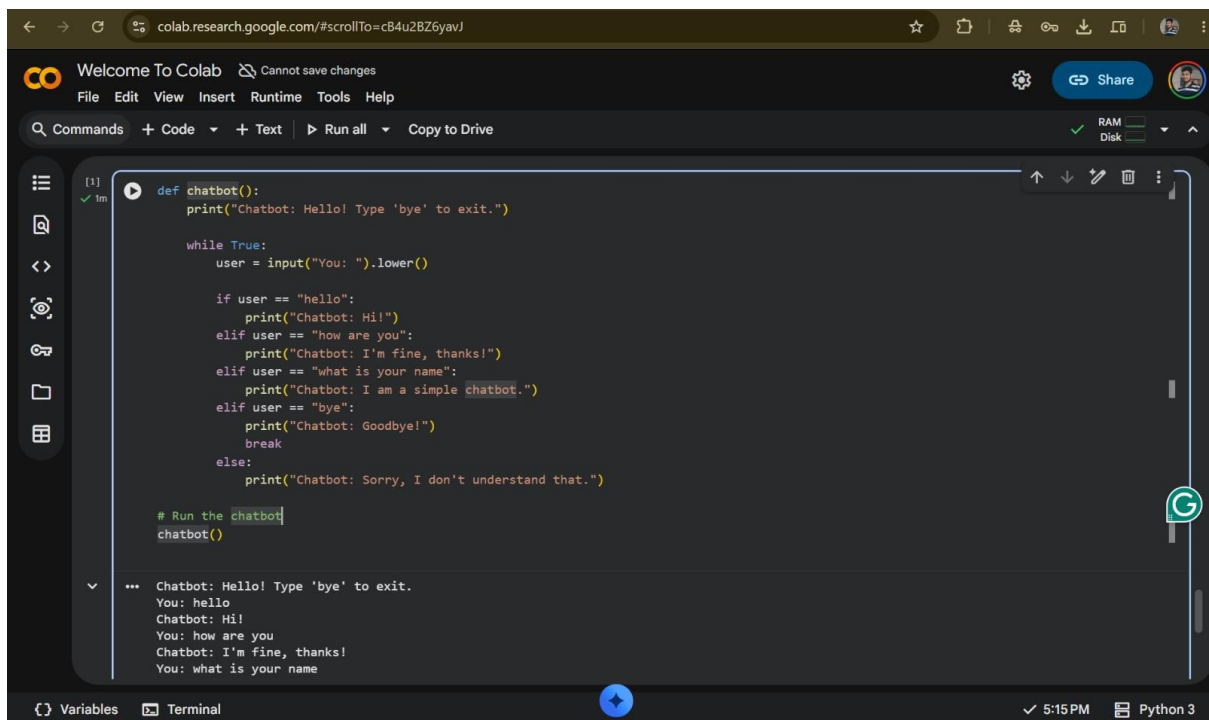
Step-by-Step Algorithm

1. Start the program.
2. Display a welcome message.
3. Read user input.
4. Convert input to lowercase.
5. Compare input with predefined commands.
6. If input is "hello", display "Hi!".
7. Else if input is "how are you", display "I'm fine, thanks!".
8. Else if input is "bye", display "Goodbye!" and terminate.
9. Otherwise display "Sorry, I don't understand that.".
10. Repeat steps 3–9 until the user exits.
11. Stop the program.

9. Source Code

```
def chatbot():  
    print("Chatbot: Hello! Type 'bye' to exit.")  
  
    while True:  
        user = input("You: ").lower()  
  
        if user == "hello":  
            print("Chatbot: Hi!")  
        elif user == "how are you":  
            print("Chatbot: I'm fine, thanks!")  
        elif user == "what is your name":  
            print("Chatbot: I am a simple chatbot.")  
        elif user == "bye":  
            print("Chatbot: Goodbye!")  
            break  
        else:  
            print("Chatbot: Sorry, I don't understand that.")  
  
chatbot()
```

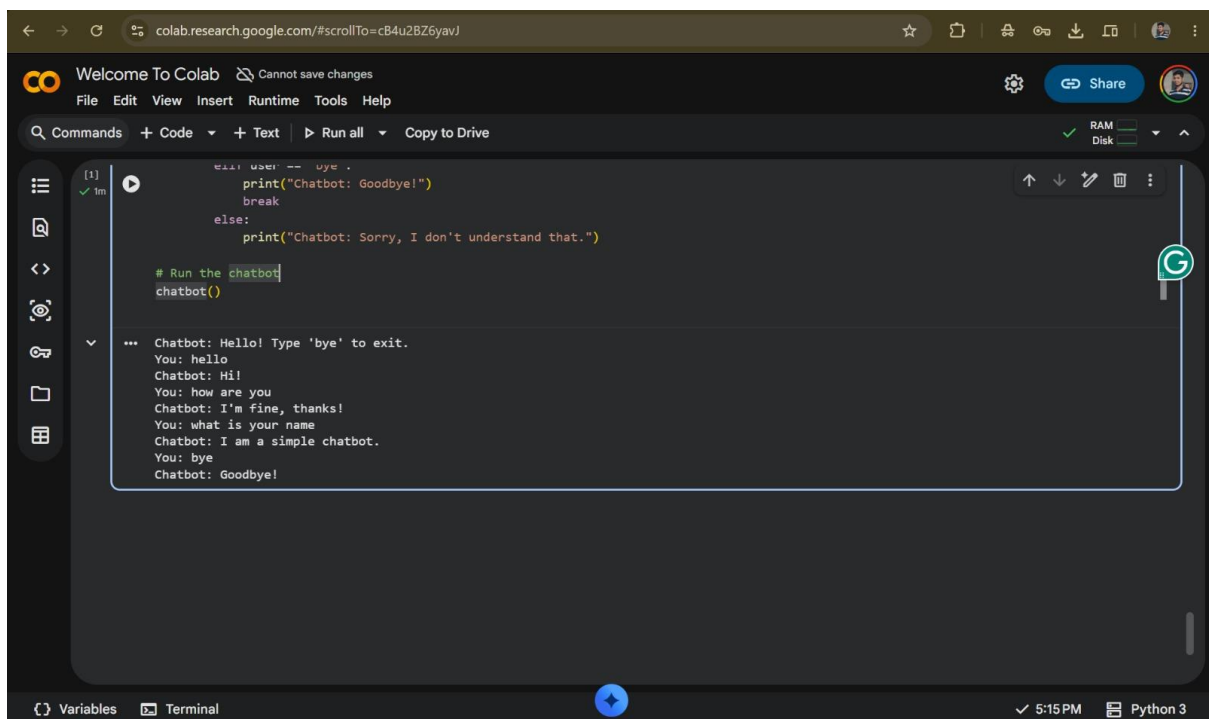
10. Output Screenshots



The screenshot shows a Google Colab notebook interface. The top bar includes the Colab logo, a "Welcome To Colab" message, and a "Cannot save changes" warning. The menu bar contains "File", "Edit", "View", "Insert", "Runtime", "Tools", and "Help". The toolbar shows "Commands", "+ Code", "+ Text", "Run all", and "Copy to Drive". The left sidebar has icons for file management and a search bar. The main code area contains a Python function `def chatbot():` that prints a greeting and enters a loop to handle user input. The output area shows the chatbot's responses to user inputs: "Hello!", "Hi!", "I'm fine, thanks!", "I am a simple chatbot.", and "Goodbye!".

```
[1] def chatbot():  
    print("Chatbot: Hello! Type 'bye' to exit.")  
  
    while True:  
        user = input("You: ").lower()  
  
        if user == "hello":  
            print("Chatbot: Hi!")  
        elif user == "how are you":  
            print("Chatbot: I'm fine, thanks!")  
        elif user == "what is your name":  
            print("Chatbot: I am a simple chatbot.")  
        elif user == "bye":  
            print("Chatbot: Goodbye!")  
            break  
        else:  
            print("Chatbot: Sorry, I don't understand that.")  
  
    # Run the chatbot  
    chatbot()
```

*** Chatbot: Hello! Type 'bye' to exit.
You: hello
Chatbot: Hi!
You: how are you
Chatbot: I'm fine, thanks!
You: what is your name



This screenshot shows the continuation of the Python chatbot script from the previous image. The code area shows the final part of the `while` loop and the `chatbot()` function call. The output area shows the chatbot's responses to the remaining user inputs: "I am a simple chatbot.", "Goodbye!", and "Sorry, I don't understand that.".

```
[1] elif user == "bye":  
    print("Chatbot: Goodbye!")  
    break  
    else:  
        print("Chatbot: Sorry, I don't understand that.")  
  
    # Run the chatbot  
    chatbot()
```

*** Chatbot: Hello! Type 'bye' to exit.
You: hello
Chatbot: Hi!
You: how are you
Chatbot: I'm fine, thanks!
You: what is your name
Chatbot: I am a simple chatbot.
You: bye
Chatbot: Goodbye!

11. Advantages

- Easy to develop and understand.
- Requires minimal resources.

- Suitable for beginners learning Python.
- Demonstrates basic AI concepts.
- Can be extended with additional features.

12. Disadvantages

- Limited to predefined responses.
- Cannot understand natural language deeply.
- No learning capability.
- Not suitable for complex conversations.

13. Applications

- Educational projects.
- Customer query handling (basic level).
- Interactive learning systems.
- Help desk prototypes.
- Beginner AI demonstrations.

14. Future Enhancements

- Add more conversation patterns.
- Integrate Natural Language Processing (NLP).
- Connect with a database for dynamic responses.
- Add voice input and output.
- Develop a graphical user interface (GUI).

15.Result

The Basic Rule-Based Chatbot was successfully developed using Python. It correctly responds to predefined user inputs such as greetings, simple questions, and exit commands. The project demonstrates the use of if-elif statements, functions, loops,

and input/output operations effectively. Thus, the chatbot achieved its intended objective successfully.

16. Conclusion

The Basic Rule-Based Chatbot was successfully developed using Python. The chatbot can interact with users through predefined responses and demonstrates the use of fundamental programming concepts such as functions, loops, conditional statements, and input/output operations. This project serves as a foundation for understanding chatbot development and can be enhanced further using advanced AI techniques.